

STUDY GUIDE · 2026

React & JavaScript From Basics to Advanced

The complete concept map — every piece of JavaScript you need for React,
then React itself from your first component to production patterns.

Tailored to the **PondyProperties** codebase
USER app · ADMIN app · React 18 + Create React App

React 18.3

Function Components + Hooks

React Router v7

Redux Toolkit

axios

socket.io

MUI + Bootstrap

ES2020+ JavaScript

What's inside

Read it top to bottom the first time. After that, use it as a reference — every concept is a self-contained card with an explanation, a runnable example, and where it shows up in your project.

1. How to use this guide & the mental model

What React actually is · SPA vs. server-rendered pages · the render loop

2. JavaScript you must know for React

let/const · arrow functions · template literals · destructuring · spread/rest · default params · ternary & short-circuit · optional chaining & nullish · array methods (map/filter/reduce...) · objects · modules · truthy/falsy · immutability · closures · promises & async/await · JSON

3. React fundamentals

Components · JSX · props · rendering lists & keys · conditional rendering · events · state with useState · controlled forms · lifting state · composition & children

4. Hooks in depth

useState deep · useEffect (lifecycle, dependencies, cleanup) · fetching data · useRef · useContext · useReducer · useMemo · useCallback · custom hooks · Rules of Hooks

5. Routing, data & state management

React Router v7 · axios patterns · environment variables · Redux Toolkit (store, slices, useSelector/useDispatch) · Context vs. Redux · real-time with socket.io

6. Advanced React & performance

Re-render model · React.memo · memo/callback in practice · lazy & Suspense (code splitting) · error boundaries · portals · forwardRef · HOCs & render props · stale closures · Strict Mode

7. Your project, mapped to concepts

Where each idea lives in USER & ADMIN · patterns you already use · things worth refactoring

8. Appendix

Rules of Hooks cheat-sheet · common mistakes · glossary · what to learn next

1 · How to use this guide & the mental model

What React actually is

React is a JavaScript library for building **user interfaces out of components**. A component is a function that takes some data (*props* and *state*) and returns a description of what the screen should look like (JSX). You never touch the DOM directly with `document.getElementById` ; instead you change data, and React figures out the minimal DOM updates for you.

THE ONE IDEA THAT UNLOCKS REACT

UI is a **function of state**: `UI = f(state)` . You don't write "when the button is clicked, find the counter span and set its text to 5." You write "the screen shows `count` ," then "clicking the button sets `count` to 5." React re-runs your component and updates the screen. Stop thinking in *steps to mutate the page*; start thinking in *what the page should be for this data*.

SPA vs. server-rendered pages

Both your USER and ADMIN apps are **Single Page Applications (SPA)** built with Create React App (`react-scripts`). The browser loads one HTML shell and a JavaScript bundle; React then renders everything and React Router swaps "pages" without a full reload. That's why navigating feels instant and why there's no separate `.html` file per screen.

The render loop (memorize this)

1. React calls your component function → it returns JSX.
2. React builds a virtual tree and compares it to the previous one (*reconciliation*).
3. Only the differences are written to the real DOM.
4. Later, some **state** changes (a `setState` call) → React re-runs the component from step 1.

A "re-render" just means "React called your function again." It is cheap and normal. It does **not** mean the whole page is thrown away.

How to read the examples

- `JS` blocks are plain JavaScript — run them in the browser console or Node.
- `JSX` blocks are React component code — they live inside `.jsx` / `.js` files.
- `In your project` notes point at real files in the USER/ADMIN apps.

2 · JavaScript you must know for React

React is "just JavaScript." Nearly every confusing React bug is actually a JavaScript misunderstanding. Master this section and React gets dramatically easier. Everything here is used constantly in your codebase.

2.1 · `let` and `const` (never `var`)

`const` = a name you won't reassign. `let` = a name you will. Both are *block-scoped* (they only exist inside the nearest `{ }`). Default to `const`; switch to `let` only when you must reassign.

```
const name = "Pondy";
let count = 0;
count = count + 1;      // ok - let can be reassigned
// name = "x";          // ERROR - const can't be reassigned

// const protects the binding, not the contents:
const user = { id: 1 };
user.id = 2;            // ok - we mutated the object, didn't reassign the name
```

WHY REACT CARES

Every hook value you'll write is a `const`: `const [count, setCount] = useState(0)`.

Understanding that `const` forbids reassignment (but not object mutation) explains why you call `setCount( ... )` instead of `count = ...`.

2.2 • Arrow functions

A shorter function syntax. Arrow functions also don't have their own `this` — they inherit it from where they're defined, which is exactly what you want inside components.

```
// classic
function add(a, b){ return a + b; }

// arrow, block body
const add = (a, b) => { return a + b; };

// arrow, implicit return (no braces, no 'return')
const add = (a, b) => a + b;
const double = n => n * 2;           // one param, parens optional
const makeUser = id => ({ id });    // return an object → wrap in ( )
```

GOTCHA

To return an object literal implicitly you must wrap it in parentheses: `() => ({ a: 1 })`. Without the parens, `{ }` is read as a function body, not an object.

In your project: every event handler is an arrow function, e.g. `const handleChange = (e) => {...}` in `ContactUs.jsx`.

2.3 • Template literals (backtick strings)

Strings with embedded expressions using ``${ }``. Cleaner than `+` concatenation and they allow multi-line strings.

```
const city = "Pondicherry";
const url = `${process.env.REACT_APP_API_URL}/property/${id}`;
const msg = `Found ${list.length} results in ${city}`;
```

You use this for every API URL, e.g.

```
axios.post(`${process.env.REACT_APP_API_URL}/contactUs`, formData) .
```

2.4 • Destructuring

Pull values out of objects/arrays into named variables in one line. This is *everywhere* in React.

```
// object destructuring
const user = { name: "Bala", phone: "99999", city: "Pondy" };
const { name, city } = user;           // name = "Bala", city = "Pondy"
const { phone: mobile } = user;        // rename → mobile = "99999"
const { role = "guest" } = user;       // default if missing

// array destructuring (position matters, names are yours)
const [first, second] = [10, 20];      // first=10, second=20
const [count, setCount] = useState(0); // THIS is why hooks look like they do

// nested + in function params
function Card({ property: { title, price } }) { /* ... */ }
```

THIS IS WHY HOOKS READ THE WAY THEY DO

`useState` returns an array `[value, setter]`. `const [count, setCount] = useState(0)` is just array destructuring. Props are read with object destructuring: `function Header({ title })`.

2.5 • Spread ... and rest ...

Same three dots, two jobs. **Spread** expands a collection; **rest** gathers leftovers into one. Spread is the backbone of immutable updates in React.

```
// SPREAD - copy + override (make a NEW object/array)
const base = { name: "Bala", city: "Pondy" };
const updated = { ...base, city: "Chennai" }; // {name:"Bala", city:"Chennai"}
const more = [ ... [1,2], 3, 4];              // [1,2,3,4]

// REST - collect the rest
const [head, ...tail] = [1,2,3,4];           // head=1, tail=[2,3,4]
function sum( ...nums){ return nums.reduce((a,b)⇒a+b, 0); }
```

This is exactly the pattern in your form handler:

```
// ContactUs.jsx - update one field without mutating the old state
const handleChange = (e) ⇒ {
  setFormData({ ...formData, [e.target.name]: e.target.value });
};
```

GOLDEN RULE

In React you **never mutate** state. You make a *copy* with spread and change the copy. `{ ... old, field: newValue }` for objects, `[ ... old, item ]` for arrays.

2.6 • Default parameters & computed property names

```
// default parameter
function greet(name = "there"){ return `Hi ${name}`; }

// computed property name - the [ ] means "use this variable as the key"
const field = "email";
const patch = { [field]: "a@b.com" }; // { email: "a@b.com" }
```

Computed keys are what make `{ [e.target.name]: e.target.value }` work for a single generic `handleChange` across every input.

2.7 • Ternary & short-circuit logic

These are how you make decisions *inside JSX*, where you can't use `if` statements.

```
// ternary: condition ? whenTrue : whenFalse
const label = isLoggedIn ? "Logout" : "Login";

// && - render the right side only if the left is truthy
error && console.log(error);

// || - fall back to the right side if the left is falsy
const name = user.name || "Guest";

// ?? (nullish) - fall back ONLY for null/undefined (not for 0 or "")
const count = value ?? 0; // keeps 0; || would replace 0 with 0 too, but ?? keeps "" and 0
```

|| vs ??

`0 || 5` is `5` (because `0` is falsy). `0 ?? 5` is `0`. Use `??` when `0`, `""`, or `false` are valid values you want to keep.

2.8 • Optional chaining `?.`

Safely read deep properties that might not exist yet — very common while data is still loading.

```
const price = property?.details?.price; // undefined instead of a crash
const first = list?.[0]; // safe array access
onClose?.(); // call only if the function exists

// Without ?. this throws if error.response is undefined:
setError(error.response?.data?.error || "Submission failed");
```

That last line is literally your ContactUs error handling made safe. Before data arrives from axios, nested fields are `undefined`; `?.` prevents "Cannot read properties of undefined."

2.9 · Array methods — the heart of rendering lists

You will use these constantly to transform API data into UI. They **return new arrays** and never mutate the original (perfect for React).

Method	Returns	Use it for
<code>map</code>	new array, same length	Turn each item into JSX / another shape
<code>filter</code>	new, shorter array	Keep only items matching a test
<code>find</code>	one item (or undefined)	Get the first match
<code>reduce</code>	a single value	Totals, grouping, folding a list into one thing
<code>some</code> / <code>every</code>	boolean	"any match?" / "all match?"
<code>includes</code>	boolean	"is this value in the array?"
<code>sort</code>	the array (mutates!)	Ordering — copy first: <code>[...arr].sort()</code>

```
const props = [
  { id:1, city:"Pondy",   price:2500000, sold:false },
  { id:2, city:"Chennai", price:4200000, sold:true  },
  { id:3, city:"Pondy",   price:1800000, sold:false },
];

// map → render a list (returns JSX per item)
props.map(p => <Card key={p.id} data={p} />);

// filter → only unsold Pondy listings
const available = props.filter(p => p.city === "Pondy" && !p.sold);

// find → one property by id
const one = props.find(p => p.id === 2);

// reduce → total value of all listings
const total = props.reduce((sum, p) => sum + p.price, 0);

// chaining - the everyday combo
const pondyTotal = props
  .filter(p => p.city === "Pondy")
  .reduce((s, p) => s + p.price, 0);
```

THE #1 REACT LIST PATTERN

```
{properties.map(p => (  
  <PropertyCard key={p._id} property={p} />  
))}
```

You'll write this hundreds of times. Note the **key** — covered in §3.4.

2.10 · Working with objects

```
const user = { name:"Bala", city:"Pondy", plan:"pro" };  
  
Object.keys(user);      // ["name", "city", "plan"]  
Object.values(user);    // ["Bala", "Pondy", "pro"]  
Object.entries(user);   // [["name", "Bala"], ["city", "Pondy"], ...]  
  
// render an object as a list of rows  
Object.entries(user).map(([key, value]) => (  
  <tr key={key}><td>{key}</td><td>{value}</td></tr>  
));
```

2.11 · ES Modules — **import** / **export**

How files share code. Two kinds of exports: **default** (one per file) and **named** (many per file).

```
// — Button.jsx —  
export default function Button(){ /* ... */ }           // default export  
export const COLORS = ["red", "blue"];                  // named export  
  
// — App.jsx —  
import Button from "./Button";                          // default: any name, no braces  
import { COLORS } from "./Button";                      // named: exact name, braces  
import Button, { COLORS } from "./Button";              // both together  
import axios from "axios";                             // from node_modules (no ./)
```

Your *store.js* shows both: **import { configureStore } from '@reduxjs/toolkit'** (named) and **import userReducer from './userSlice'** (default).

2.12 · Truthy / falsy

Any value can be treated as true/false in a condition. The **falsy** values are exactly: `false`, `0`, `""` (empty string), `null`, `undefined`, `NaN`. Everything else is truthy — including `[]` and `{}`.

CLASSIC REACT BUG

```
{list.length && <List items={list} />} // BUG: when length is 0,  
                                         // React renders the number 0 on screen!  
{list.length > 0 && <List items={list} />} // FIX: force a real boolean
```

2.13 · Immutability (the React survival rule)

Treat state as read-only. Never push into an array or set a property on an object that's in state — always create a new value. React decides whether to re-render by checking if the *reference* changed, so a mutated-in-place object looks "unchanged" to React.

```
// ❌ mutation - React may not see the change  
todos.push(newTodo);  
user.name = "New";  
  
// ✅ new references  
setTodos([... todos, newTodo]); // add  
setTodos(todos.filter(t => t.id !== id)); // remove  
setTodos(todos.map(t => t.id === id ? {...t, done:true} : t)); // update one  
setUser({ ...user, name: "New" }); // change a field
```

2.14 · Closures (why this matters for hooks)

A closure is a function "remembering" the variables around it when it was created. Every render of a component creates fresh functions that close over *that render's* props and state. This explains a whole class of "why is my value stale?" bugs.

```
function makeCounter(){  
  let count = 0;  
  return () => ++count; // this inner fn "closes over" count  
}  
  
const next = makeCounter();  
next(); next(); // 1, then 2 - count persists via the closure
```

WHERE THIS BITES YOU

Inside `setTimeout` or an event handler, `count` is the value from the render that created it — it won't magically update. The fix is the *functional updater*: `setCount(c => c + 1)` (see §4.1).

2.15 · Promises & `async / await`

Network calls take time. A **Promise** represents a value that will arrive later. `async/await` is the readable way to work with promises — it lets asynchronous code read top-to-bottom like normal code.

```
// Promise style (.then / .catch)
axios.get("/api/props")
  .then(res => setList(res.data))
  .catch(err => setError(err.message));

// async/await style (what your code uses) - cleaner
const loadProps = async () => {
  try {
    const res = await axios.get("/api/props"); // pause here until it resolves
    setList(res.data);
  } catch (err) {
    setError(err.message); // runs if it rejects
  } finally {
    setLoading(false); // always runs
  }
};
```

RULES OF THUMB

- `await` only works inside an `async` function.
- `await someAsync()` gives you the resolved value, not a Promise.
- Always wrap `await` in `try/catch` for network calls (your `ContactUs.jsx` does this correctly).

This is exactly your `handleSubmit = async (e) => { ... await axios.post(...) ... }` pattern.

2.16 · JSON

```
const obj = { id: 1, city: "Pondy" };
const text = JSON.stringify(obj); // object → string (for storage/network)
const back = JSON.parse(text); // string → object

// common with localStorage
localStorage.setItem("user", JSON.stringify(user));
const saved = JSON.parse(localStorage.getItem("user") || "null");
```

axios parses JSON for you automatically — `res.data` is already a JS object, no `JSON.parse` needed.

3 · React fundamentals

Now the React-specific building blocks. Everything here uses the JavaScript from Part 2.

3.1 · Components

A component is a function whose name is **Capitalized** and that returns JSX. That's the whole definition. You compose small components into bigger ones.

```
function Welcome() {  
  return <h1>Welcome to PandyProperties</h1>;  
}  
  
// used like an HTML tag:  
export default function App() {  
  return (  
    <div>  
      <Welcome />  
      <Welcome />  
    </div>  
  );  
}
```

Your *App.js* is exactly this: it returns `<Header />` , `<BannerCarousel />` , `<Carousel />` , etc. composed together.

MUST-KNOW RULES

- Component names **must start with a capital letter** (`Header` , not `header`) or React treats it as a plain HTML tag.
- A component must return **one** root element. Wrap siblings in a `<div>` or a Fragment `<>...` .

3.2 · JSX

JSX looks like HTML but is really JavaScript. It compiles to function calls. A few differences from HTML you must remember:

HTML	JSX	Why
<code>class</code>	<code>className</code>	<code>class</code> is a reserved JS word
<code>for</code>	<code>htmlFor</code>	same reason
<code>onclick</code>	<code>onClick</code>	camelCase events
<code>style="color:red"</code>	<code>style={{color:'red'}}</code>	style is an object
<code>
</code>	<code>
</code>	tags must self-close

```
const city = "Pondy";
const jsx = (
  <div className="card" style={{ padding: 12, fontWeight: "bold" }}>
    <h2>Properties in {city}</h2>      {/* { } embeds JS expressions */}
    <p>{2 + 2} results</p>
    {/* this is how you write a comment inside JSX */}
  </div>
);
```

INSIDE { } GOES AN EXPRESSION

You can put any JS expression in braces: a variable, `a + b`, a ternary, a `.map()`. You **cannot** put statements like `if` or `for` — that's why you use ternaries and `.map()` instead.

Note in `App.js`: `style={{fontFamily:"Inter, sans-serif"}}` — the outer `{}` means "JS here," the inner `{}` is the object.

3.3 · Props — passing data into components

Props are the arguments to a component. Data flows **down**, parent → child. Props are read-only; a child never edits its own props.

```
// child - reads props (destructured)
function PropertyCard({ title, price, city }) {
  return (
    <div className="card">
      <h3>{title}</h3>
      <p>{city} - ₹{price.toLocaleString("en-IN")}</p>
    </div>
  );
}

// parent - passes props like HTML attributes
<PropertyCard title="2BHK Flat" price={2500000} city="Pondy" />

// strings use quotes; anything else uses { }
<PropertyCard title="Villa" price={price} city={selectedCity} sold={false} />
```

PASSING FUNCTIONS DOWN AS PROPS

Children talk back to parents by calling a function the parent passed: `<SearchBar onSearch={handleSearch} />`. The child calls `onSearch(value)`; the parent decides what happens. This is how "data down, events up" works.

3.4 · Rendering lists & the `key` prop

Render arrays with `.map()`. Each element needs a stable, unique `key` so React can track items across re-renders.

```
function PropertyList({ properties }) {
  return (
    <div>
      {properties.map(p => (
        <PropertyCard key={p._id} title={p.title} price={p.price} />
      ))}
    </div>
  );
}
```

KEY RULES

- Use a **stable unique id** from your data (Mongo's `_id` is perfect).
- **Avoid the array index** as key when the list can reorder, filter, or have items inserted/removed — it causes subtle UI bugs (wrong item state, wrong inputs).
- The `key` goes on the outermost element returned by `map`, and it's not readable as a prop inside the child.

3.5 · Conditional rendering

```
function Status({ loading, error, data }) {
  if (loading) return <Spinner />; // early return
  if (error) return <p className="text-danger">{error}</p>;

  return (
    <div>
      {data.length === 0
        ? <p>No properties found</p> // ternary for either/or
        : <PropertyList properties={data} />}

      {user && <WelcomeBanner name={user.name} />} // && for "show or nothing"
    </div>
  );
}
```

Three tools: **early return** (whole-component branches), **ternary** (either/or inside JSX), **&&** (show something or nothing). Remember the `length > 0` guard from §2.12.

3.6 · Handling events

```
function SearchBar() {
  const handleClick = () => console.log("clicked");

  return (
    <form onSubmit={e => { e.preventDefault(); /* ... */ }}>
      <input onChange={e => console.log(e.target.value)} />
      <button onClick={handleClick}>Search</button>
    </form>
  );
}
```

TWO BEGINNER TRAPS

- Pass the function, don't call it: `onClick={handleClick}`  `onClick={handleClick()}`  (that runs it immediately during render).
- To pass arguments, wrap in an arrow: `onClick={() => deleteItem(id)}`.
- Call `e.preventDefault()` in form submit handlers to stop the browser's full-page reload — your *ContactUs.jsx* does this.

3.7 · State with `useState`

State is data that changes over time and, when it changes, re-renders the component. `useState` gives you a value and a setter.

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0); // 0 is the initial value

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

MENTAL MODEL

1. Calling `setCount` tells React "this value changed."
2. React re-runs `Counter()` ; this time `count` is the new value.
3. The new JSX is diffed and the DOM updates.

STATE UPDATES ARE ASYNCHRONOUS

After `setCount(5)` , reading `count` on the very next line still gives the *old* value. The new value shows up on the next render. Don't rely on it changing immediately.

3.8 · Controlled forms

A "controlled" input is one whose value comes from state. React is the single source of truth for the field. This is the standard pattern and exactly how your forms work.

```
function ContactForm() {
  const [formData, setFormData] = useState({ name:"", email:"", message:"" });

  const handleChange = (e) =>
    setFormData({ ...formData, [e.target.name]: e.target.value });

  const handleSubmit = async (e) => {
    e.preventDefault();
    await axios.post(`${process.env.REACT_APP_API_URL}/contactUs`, formData);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" value={formData.name} onChange={handleChange} />
      <input name="email" value={formData.email} onChange={handleChange} />
      <textarea name="message" value={formData.message} onChange={handleChange} />
      <button>Send</button>
    </form>
  );
}
```

THE TRICK THAT MAKES ONE HANDLER SERVE EVERY FIELD

Give each input a `name` that matches a key in state, then use the computed key `[e.target.name]`. One `handleChange` updates any field. This is precisely your *ContactUs.jsx* design.

3.9 · Lifting state up

When two components need the same data, move the state to their nearest common parent and pass it down as props. The parent "owns" the state; children receive values and change-callbacks.

```
function SearchPage() {
  const [query, setQuery] = useState(""); // owned here
  return (
    <
      <SearchInput value={query} onChange={setQuery} /> // child sets it
      <Results query={query} /> // sibling reads it
    </>
  );
}
```

3.10 • Composition & `children`

Wrap content with a component using the special `children` prop. Great for layout shells, cards, and modals.

```
function Card({ title, children }) {
  return (
    <div className="card shadow-lg p-4">
      <h3>{title}</h3>
      <div className="card-body">{children}</div>
    </div>
  );
}

// usage - whatever you nest becomes `children`
<Card title="Contact">
  <p>Reach us anytime.</p>
  <ContactForm />
</Card>
```

This is the idiomatic React alternative to copy-pasting the same wrapper markup (like the Bootstrap `card shadow-lg` blocks repeated across your components).

4 · Hooks in depth

Hooks are functions starting with `use` that let function components have state, side effects, and more. This is the core of modern React and 90% of what you write day to day.

4.1 · `useState` — the details that trip people up

```
const [value, setValue] = useState(initial);

// 1) Functional updater - use when new state depends on old state
setCount(c => c + 1);           // safe even in loops/timeouts/batched updates

// 2) Lazy initial state - expensive init runs ONCE, not every render
const [data] = useState(() => JSON.parse(localStorage.getItem("cart")));

// 3) Updating objects/arrays - always a NEW reference (spread)
setUser(u => ({ ...u, city: "Chennai" }));
setList(l => [...l, newItem]);
```

WHEN TO USE THE FUNCTIONAL UPDATER

If your next value is based on the current one — counters, toggles, appending — use `setX(prev => ...)`. It avoids stale-closure bugs (§2.14) because React hands you the latest value.

4.2 • `useEffect` — synchronizing with the outside world

Effects run **after** render. Use them for things outside React's world: fetching data, subscriptions, timers, manually touching the DOM, socket connections. The dependency array controls *when* the effect re-runs.

```
import { useEffect, useState } from "react";

function Properties() {
  const [list, setList] = useState([]);

  // [] → run ONCE after first render (like "on mount")
  useEffect(() => {
    let active = true;
    axios.get(`${process.env.REACT_APP_API_URL}/properties`)
      .then(res => { if (active) setList(res.data); });
    return () => { active = false; }; // cleanup - avoids setting state after unmount
  }, []);

  return <PropertyList properties={list} />;
}
```

The three dependency-array shapes

Code	Runs...
<code>useEffect(fn)</code> (no array)	after every render — rarely what you want
<code>useEffect(fn, [])</code>	once , after the first render (mount)
<code>useEffect(fn, [id, query])</code>	on mount and whenever <code>id</code> or <code>query</code> changes

Cleanup functions

Return a function to tear down. It runs before the effect re-runs and when the component unmounts. Essential for timers, event listeners, and sockets.

```
useEffect(() => {
  const socket = io(process.env.REACT_APP_API_URL);
  socket.on("newLead", lead => setLeads(prev => [lead, ...prev]));
  return () => socket.disconnect(); // cleanup - prevents duplicate connections
}, []);
```

THE DEPENDENCY-ARRAY MISTAKES THAT CAUSE 90% OF EFFECT BUGS

- **Missing dependency** → effect uses a stale value. Include every prop/state/function it reads.
- **Object/array/function in deps** → it's a new reference each render, so the effect runs every time. Fix with `useMemo` / `useCallback` (§4.6–4.7) or move it inside the effect.
- **Infinite loop** → the effect sets state that's also in its dependency array. Rethink the dependency, or use a functional updater.
- **Fetching without cleanup** → "Can't perform a React state update on an unmounted component." Use the `active` flag or an `AbortController`.

YOU OFTEN DON'T NEED AN EFFECT

Don't use an effect to compute a value from props/state — just compute it during render (*derived state*). Effects are for *side effects* (network, subscriptions, DOM), not for transforming data you already have.

4.3 · Fetching data (the complete pattern)

```
function usePropertiesPage(city) {
  const [data, setData] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();
    setLoading(true);
    setError(null);

    axios.get(`${process.env.REACT_APP_API_URL}/properties`, {
      params: { city },
      signal: controller.signal,
    })
      .then(res => setData(res.data))
      .catch(err => { if (err.name !== "CanceledError") setError(err.message); })
      .finally(() => setLoading(false));

    return () => controller.abort(); // cancel if city changes / unmount
  }, [city]); // re-fetch when city changes

  return { data, loading, error };
}
```

Loading + error + data is the universal trio. Every data screen in USER/ADMIN benefits from this shape.

4.4 • `useRef` — a value that survives renders without causing one

Two uses: (1) grab a DOM node, (2) hold a mutable value that *doesn't* trigger re-renders (timers, previous values, flags).

```
function SearchBox() {
  const inputRef = useRef(null);

  useEffect(() => { inputRef.current.focus(); }, []); // focus on mount

  return <input ref={inputRef} />;
}

// mutable value that persists but doesn't re-render:
const timer = useRef(null);
timer.current = setTimeout(...); // changing .current never re-renders
```

REF VS STATE

Changing **state** re-renders; changing a **ref** does not. Use state for anything shown on screen; use a ref for behind-the-scenes values (a socket instance, an interval id, "did I already run this?").

4.5 • `useContext` — share data without prop-drilling

When many components deep in the tree need the same value (current user, theme, cart), Context lets them read it directly instead of passing props through every layer.

```
// 1) create
const AuthContext = createContext(null);

// 2) provide (near the top of the app)
function App() {
  const [user, setUser] = useState(null);
  return (
    <AuthContext.Provider value={{ user, setUser }}>
      <Routes />
    </AuthContext.Provider>
  );
}

// 3) consume (anywhere below, no props needed)
function Navbar() {
  const { user } = useContext(AuthContext);
  return <span>{user ? user.name : "Login"}</span>;
}
```

TRADE-OFF

Every component reading a context re-renders when its `value` changes. Keep fast-changing values out of one giant context, or split contexts. For app-wide, frequently-updated state, Redux Toolkit (§5.4) is often the better tool — which is why your project uses it.

4.6 • `useReducer` — structured state for complex logic

When state has many sub-values or the next state depends on an "action," a reducer is cleaner than juggling several `useState` s. (It's also the mental model behind Redux.)

```
function reducer(state, action) {
  switch (action.type) {
    case "loading": return { ...state, loading: true, error: null };
    case "success": return { loading: false, error: null, data: action.payload };
    case "error":   return { ...state, loading: false, error: action.payload };
    default:       return state;
  }
}

function List() {
  const [state, dispatch] = useReducer(reducer, { loading:false, error:null, data:[] });

  useEffect(() => {
    dispatch({ type: "loading" });
    axios.get("/api/props")
      .then(res => dispatch({ type: "success", payload: res.data }))
      .catch(err => dispatch({ type: "error",   payload: err.message }));
  }, []);

  // state.loading / state.error / state.data
}
```

4.7 • `useMemo` & `useCallback` — memoization

Both cache something between renders so you don't recompute or recreate it unnecessarily. Reach for them to fix a measured performance problem or to stabilize a dependency — not by default.

```
// useMemo - cache a computed VALUE; recompute only when deps change
const sortedProps = useMemo(
  () => [...props].sort((a, b) => b.price - a.price),
  [props]
);

// useCallback - cache a FUNCTION identity; stable unless deps change
const handleSelect = useCallback((id) => {
  setSelected(id);
}, []); // same function reference across renders
```

WHEN EACH EARNS ITS KEEP

- `useMemo` : an expensive calculation (big sort/filter/reduce) on every render, or to keep an object/array reference stable so a child / effect doesn't re-run.
- `useCallback` : pass a stable function to a `React.memo` child, or to an effect's dependency array.
- Don't sprinkle them everywhere — they add overhead and clutter. Profile first.

4.8 • Custom hooks — reuse stateful logic

A custom hook is just a function starting with `use` that calls other hooks. Extract repeated logic (fetching, form handling, toggles) once and reuse it everywhere.

```
// useFetch.js - reusable data fetching
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let active = true;
    setLoading(true);
    axios.get(url)
      .then(res => active && setData(res.data))
      .catch(err => active && setError(err.message))
      .finally(() => active && setLoading(false));
    return () => { active = false; };
  }, [url]);

  return { data, loading, error };
}

// any component:
const { data, loading, error } = useFetch(`${API}/properties`);
```

Your USER/ADMIN apps repeat the fetch-loading-error pattern in dozens of files. A single `useFetch` (or a small `useApi`) would remove a lot of duplication.

4.9 · The Rules of Hooks (break these and React breaks)

NON-NEGOTIABLE

1. **Only call hooks at the top level** of a component or custom hook. Never inside `if`, loops, or nested functions. React relies on hooks being called in the same order every render.
2. **Only call hooks from React functions** — components or other hooks, never plain functions or event handlers.
3. Custom hooks **must start with** `use` so the linter can check them.

// ❌ conditional hook – order changes between renders → crash

```
if (isLoggedIn) { const [x] = useState(0); }
```

// ✅ hook always runs; put the condition inside

```
const [x, setX] = useState(0);  
if (isLoggedIn) { /* use x */ }
```

5 · Routing, data & state management

The libraries that turn components into a real app — the exact ones in your `package.json`.

5.1 · React Router v7

Maps URLs to components so your SPA has multiple "pages." Your apps use `react-router-dom` v7.

```
import { BrowserRouter, Routes, Route, Link, useNavigate, useParams } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/properties" element={<PropertyList />} />
        <Route path="/property/:id" element={<PropertyDetail />} /> { /* :id = URL param */ }
        <Route path="*" element={<NotFound />} /> { /* catch-all */ }
      </Routes>
    </BrowserRouter>
  );
}
```

Navigating & reading params

```
// declarative link (no page reload)
<Link to="/properties">Browse</Link>

// programmatic navigation (after a login/submit)
const navigate = useNavigate();
navigate("/dashboard");
navigate(-1); // go back

// read the :id from /property/42
const { id } = useParams();
useEffect(() => { /* fetch property #id */ }, [id]);
```

Your `ContactUs.jsx` already imports `useNavigate` from `react-router-dom` — same idea.

5.2 · axios & environment variables

axios is your HTTP client. In Create React App, any env var prefixed `REACT_APP_` is available as `process.env.REACT_APP_*` at build time.

```
// .env (project root, next to package.json)
REACT_APP_API_URL=https://vps.ppcpondy.com/api

// usage
const API = process.env.REACT_APP_API_URL;
const { data } = await axios.get(`${API}/properties`);
await axios.post(`${API}/contactUs`, formData);

// a shared instance keeps config in one place (recommended)
// api.js
export const api = axios.create({
  baseURL: process.env.REACT_APP_API_URL,
  timeout: 15000,
});
// then: api.get("/properties") - no repeating the base URL
```

CRA ENV VAR GOTCHAS

- Must start with `REACT_APP_` or CRA ignores it.
- They're baked in at **build** time — restart `npm start` after editing `.env`.
- They're visible in the shipped bundle — **never** put secrets (API keys, passwords) in `REACT_APP_*`. Secrets belong on the backend.

5.3 · axios interceptors (auth tokens in one place)

```
api.interceptors.request.use(config => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

api.interceptors.response.use(
  res => res,
  err => {
    if (err.response?.status === 401) { /* redirect to login */ }
    return Promise.reject(err);
  }
);
```

Given your token-based admin auth, an interceptor is the clean place to attach the token to every request instead of repeating it per call.

5.4 · Redux Toolkit — your global state solution

Redux Toolkit (RTK) holds app-wide state in one *store* so any component can read/update it without prop-drilling. Your *red/store.js* already wires this up. The three pieces:

1) The store (you have this)

```
// red/store.js
import { configureStore } from "@reduxjs/toolkit";
import userReducer from "../userSlice";

export const store = configureStore({
  reducer: { user: userReducer }, // state.user comes from userSlice
});
```

2) A slice — state + reducers together

```
// red/userSlice.js
import { createSlice } from "@reduxjs/toolkit";

const userSlice = createSlice({
  name: "user",
  initialState: { phoneNumber: null, isLoggedIn: false },
  reducers: {
    setUser(state, action) { // RTK lets you "mutate" (Immer copies safely)
      state.phoneNumber = action.payload;
      state.isLoggedIn = true;
    },
    logout(state) {
      state.phoneNumber = null;
      state.isLoggedIn = false;
    },
  },
});

export const { setUser, logout } = userSlice.actions; // action creators
export default userSlice.reducer;
```

3) Provide the store, then read/dispatch

```
// index.js - wrap the app once
import { Provider } from "react-redux";
import { store } from "../red/store";
root.render(<Provider store={store}><App /></Provider>);

// any component:
import { useSelector, useDispatch } from "react-redux";
import { setUser, logout } from "../red/userSlice";

function Profile() {
  const phone    = useSelector(state => state.user.phoneNumber); // READ
  const dispatch = useDispatch();                                // WRITE

  return (
    <div>
      <p>{phone}</p>
      <button onClick={() => dispatch(logout())}>Logout</button>
    </div>
  );
}
```

THE ONE-LINE SUMMARY

Slice defines state + how it changes. **useSelector** reads a piece of it. **useDispatch** sends an action to change it. RTK's Immer lets you write "mutating" code in reducers that's actually immutable under the hood.

5.5 • Context vs. Redux — which to use

Use Context when...	Use Redux Toolkit when...
Rarely-changing global values (theme, current user, language)	Frequently-changing state shared widely (cart, live leads, filters)
Small app, few consumers	Many actions, complex updates, you want devtools/time-travel
No extra library wanted	You want a predictable, testable update flow

Your project already standardized on Redux Toolkit — keep new global state in slices for consistency.

5.6 • Real-time with socket.io

Your USER app depends on `socket.io-client`. Sockets belong in an effect with cleanup so you don't stack up connections.

```
import { io } from "socket.io-client";

function LeadsCenter() {
  const [leads, setLeads] = useState([]);

  useEffect(() => {
    const socket = io(process.env.REACT_APP_API_URL);

    socket.on("connect", () => console.log("connected", socket.id));
    socket.on("newLead", (lead) => setLeads(prev => [lead, ...prev])); // functional update!

    return () => socket.disconnect(); // cleanup on unmount
  }, []);

  return <LeadList leads={leads} />;
}
```

TWO SOCKET PITFALLS

- Forgetting `disconnect()` in cleanup → duplicated listeners and memory leaks (especially with React 18 Strict Mode double-invoking effects in dev).
- Using `setLeads([lead, ...leads])` instead of the functional form — `leads` is stale inside the once-created handler. Always `setLeads(prev => ...)`.

6 · Advanced React & performance

The patterns that separate "it works" from "it scales." You won't need all of these daily, but knowing they exist tells you what to reach for.

6.1 · The re-render model (what actually triggers renders)

A component re-renders when: (1) its **own state** changes, (2) its **parent** re-renders, or (3) a **context** it consumes changes. Note #2: a parent re-render re-renders *all* children by default, even if their props didn't change. That's usually fine — renders are cheap — but it's the root of most "why is this slow" questions.

DON'T OPTIMIZE BLINDLY

Re-rendering $\neq$ re-touching the DOM. React still only updates the DOM where output actually differs. Measure with the React DevTools Profiler before adding memoization.

6.2 · `React.memo` — skip re-rendering unchanged children

```
const PropertyCard = React.memo(function PropertyCard({ property }) {  
  // re-renders only when the `property` prop actually changes  
  return <div className="card">{property.title}</div>;  
});
```

MEMO ONLY HELPS IF PROPS ARE STABLE

`React.memo` does a shallow prop comparison. If you pass a fresh object/array/function each render (`onClick={() => ...}` , `style={{...}}`), the props "changed" every time and memo does nothing. Pair it with `useCallback` / `useMemo` for the props.

6.3 · Code splitting — `lazy` + `Suspense`

Your apps are large (200+ components). Loading everything up front makes the first paint slow.

`React.lazy` splits a component into its own chunk that downloads on demand.

```
import { lazy, Suspense } from "react";  
const AdminDashboard = lazy(() => import("./AdminDashboard"));  
  
<Suspense fallback=<Spinner />>  
  <AdminDashboard />  
</Suspense>
```

Lazy-loading heavy, rarely-visited routes (admin reports, PDF/Excel exports via `jspdf` / `xlsx`) can noticeably shrink your initial bundle and speed up load — a real win for both apps.

6.4 • Error boundaries

A JavaScript error in one component shouldn't blank the whole app. An error boundary catches render errors in its subtree and shows a fallback. (This is the one thing still requiring a class component, or the `react-error-boundary` library.)

```
class ErrorBoundary extends React.Component {
  state = { hasError: false };
  static getDerivedStateFromError() { return { hasError: true }; }
  componentDidCatch(error, info) { /* log to a service */ }
  render() {
    return this.state.hasError
      ? <p>Something went wrong. Please refresh.</p>
      : this.props.children;
  }
}

// wrap risky areas (or the whole router)
<ErrorBoundary><Routes /></ErrorBoundary>
```

6.5 • Portals — render outside the parent DOM node

Modals, toasts, and tooltips often need to escape overflow/z-index traps. A portal renders children into a different DOM node while keeping them in the React tree.

```
import { createPortal } from "react-dom";
function Modal({ children }) {
  return createPortal(
    <div className="modal-backdrop">{children}</div>,
    document.body
  );
}
```

Libraries you use (`react-toastify` , `sweetalert2` , MUI dialogs) rely on this internally.

6.6 • `forwardRef` — let a parent reach a child's DOM node

```
const Input = React.forwardRef(function Input(props, ref) {
  return <input ref={ref} { ... props } />;
});

// parent can now focus it
const ref = useRef(null);
<Input ref={ref} />;
ref.current.focus();
```

6.7 · HOCs & render props (older reuse patterns)

Before hooks, logic was shared via **Higher-Order Components** (a function taking a component, returning an enhanced one) and **render props**. You'll meet them in older code and libraries; for new code, prefer custom hooks (§4.8).

```
// HOC
const withLoading = (Component) => ({ loading, ...props }) =>
  loading ? <Spinner /> : <Component {...props} />;

// render prop
<DataFetcher url="/api/props" render={({data}) => <List items={data} />} />
```

6.8 · Stale closures & the functional updater (recap with teeth)

```
function Timer() {
  const [count, setCount] = useState(0);
  useEffect(() => {
    const id = setInterval(() => {
      // ❌ count is frozen at 0 forever (closure from first render)
      // setCount(count + 1);
      setCount(c => c + 1); // ✅ always uses the latest value
    }, 1000);
    return () => clearInterval(id);
  }, []); // empty deps - effect runs once, so `count` never updates inside it
  return <p>{count}</p>;
}
```

6.9 · React 18 Strict Mode (why effects run twice in dev)

In development, React 18 intentionally mounts, unmounts, and re-mounts components once to surface missing cleanup. If your effect isn't idempotent (no cleanup), you'll see doubled API calls or socket connections *in dev only*. The fix is always: **write proper cleanup**. Production runs once.

6.10 · A quick word on class components

You'll see class components in older tutorials and a few legacy files. Mapping to hooks:

Class	Hook equivalent
<code>this.state</code> / <code>this.setState</code>	<code>useState</code>
<code>componentDidMount</code>	<code>useEffect(fn, [])</code>
<code>componentDidUpdate</code>	<code>useEffect(fn, [deps])</code>
<code>componentWillUnmount</code>	cleanup <code>return</code> in <code>useEffect</code>

Write new components as functions with hooks. Keep classes only for error boundaries.

7 · Your project, mapped to concepts

Where each idea already lives in PondyProperties — so you can go read real, working examples right after learning the concept.

Concept	Where to see it in your code
Component composition	<code>USER/src/App.js</code> — <code>Header</code> , <code>BannerCarousel</code> , <code>Carousel</code> , <code>Ads</code> , <code>FrontFooter</code> assembled together
Controlled forms + <code>useState</code> + spread update	<code>USER/src/Components/ContactUs.jsx</code> — <code>formData</code> + <code>handleChange</code> with <code>[e.target.name]</code>
async/await + axios + try/catch	<code>ContactUs.jsx</code> — <code>handleSubmit</code> posting to <code>/contactUs</code>
Env-based API URL	Every axios call: <code>`\${process.env.REACT_APP_API_URL}/...`</code>
Routing / navigation	<code>useNavigate</code> imported in <code>ContactUs.jsx</code> ; route table in your top-level router
Redux Toolkit store	<code>USER/src/red/store.js</code> + <code>red/userSlice.js</code> (holds <code>phoneNumber</code>)
Lists & keys	Any table/list screen in <code>ADMIN/src</code> (e.g. <code>AllBills.jsx</code> , <code>BuyersMenu.jsx</code>) mapping API rows
Real-time	<code>socket.io-client</code> in the USER app (leads/notifications)
Third-party via portals	<code>react-toastify</code> , <code>sweetalert2</code> , MUI dialogs
Heavy libs to lazy-load	<code>jspdf</code> , <code>jspdf-autotable</code> , <code>xlsx</code> , <code>html2pdf.js</code> in ADMIN exports

High-value refactors to practice these concepts

1. **Extract a `useFetch` / `useApi` custom hook** (§4.8). The loading/error/data trio is duplicated across dozens of files — centralize it.
2. **Create one axios instance with an auth interceptor** (§5.3) instead of repeating `process.env.REACT_APP_API_URL` and the token everywhere.
3. **Lazy-load export-heavy admin screens** (§6.3) that pull in `jspdf` / `xlsx` to shrink the initial bundle.
4. **Add an error boundary** (§6.4) around your router so one broken screen can't blank the app.
5. **Audit socket effects** (§5.6) for `disconnect()` cleanup and functional state updates.
6. **Replace array-index keys** (§3.4) with Mongo `_id` in lists that filter or reorder.

8 · Appendix

8.1 · Rules of Hooks — cheat sheet

- Call hooks only at the **top level** — never in `if` /loops/nested functions.
- Call hooks only from **components or custom hooks**.
- Custom hooks start with `use`.
- `useEffect` deps must list **everything the effect reads**.
- Base new state on old state with the **functional updater**: `setX(prev => ...)`.
- Never mutate state — always create a **new object/array**.
- Return a **cleanup** from effects that subscribe/time/connect.

8.2 · Common mistakes & fixes

Symptom	Cause	Fix
"0" or "false" appears on screen	<code>{count && ...}</code> with falsy left side	<code>{count > 0 && ...}</code>
UI doesn't update after change	Mutated state in place	Copy with spread, then set
Infinite re-render loop	Effect sets state that's in its deps; or object literal in deps	Fix deps; memoize; functional update
Stale value in timer/handler	Closure captured old state	<code>setX(prev => ...)</code> / add to deps
Two API calls / double socket in dev	Strict Mode + missing cleanup	Return cleanup from effect
"Cannot read properties of undefined"	Reading data before it loads	Optional chaining <code>?.</code> + loading guard
Handler runs on render, not click	<code>onClick={fn()}</code>	<code>onClick={fn}</code> or <code>onClick={() => fn(arg)}</code>
Wrong item edited in a list	Array index used as <code>key</code>	Use a stable id

8.3 · Glossary

JSX	HTML-like syntax that compiles to JS; what components return
Component	Capitalized function returning JSX
Props	Read-only inputs passed parent → child
State	Data that changes and triggers re-render
Hook	<code>use*</code> function adding state/effects to function components
Effect	Side-effect code that runs after render (fetch, subscribe)
Reconciliation	React diffing new vs old tree to update the DOM minimally
Reducer	Pure function <code>(state, action) ⇒ newState</code>
Slice	RTK bundle of state + reducers + actions for one feature
Selector	Function reading a piece of Redux state (<code>useSelector</code>)
Memoization	Caching a value/function to avoid recomputing
Portal	Render children into a different DOM node
SPA	Single Page App — one HTML shell, JS renders everything

8.4 · A sensible learning order (check them off)

1. JS: destructuring, spread, arrow functions, array methods, async/await (Part 2)
2. Components, JSX, props, lists+keys, conditional rendering (Part 3)
3. `useState` + controlled forms — rebuild your ContactUs form from scratch
4. `useEffect` + axios — build a list screen with loading/error/data
5. React Router — wire up 3–4 pages with a detail route (`:id`)
6. Redux Toolkit — read/dispatch your existing `userSlice`
7. Custom hooks — extract `useFetch` and use it in two screens
8. Performance — memo/useMemo/useCallback + the Profiler, only where measured
9. Advanced — lazy/Suspense, error boundaries, portals

FINAL ADVICE

Don't just read — **rebuild pieces of your own app**. Every concept here maps to something already in PondyProperties. Pick one screen, recreate it from the concept up, and the ideas will stick far better than any tutorial. You already have the best practice project available: the real one you work in.